

Interview Practice Problems

Five warm-up + coding problems focused on scheduling, rate limiting, dependency graphs, counting overlap, and priority processing.

Problem 1: Server Jobs

Warm-up

You are given a list of jobs that need to run on a server.

Each job has:

```
[start_time, end_time]
```

For example:

```
[1, 5]  
[6, 9]  
[4, 7]
```

Two jobs conflict if they are running at the same time.

Write a function that returns true if any two jobs overlap. Otherwise, return false.

Code Problem

Now assume you can have multiple identical servers.

A server can only run one job at a time. Once a job finishes, that server can immediately be used by another job.

Given all of the jobs, return the minimum number of servers required so that every job can run at its scheduled time.

Example:

```
jobs = {  
    {1, 5},  
    {2, 6},  
    {7, 9}  
}
```

Output:

```
2
```

The first two jobs overlap, so they need different servers. The third job can reuse one of those servers.

Problem 2: Website Request Limit

Warm-up

A website records the time of every request made by a user.

You are given the timestamps for one user in increasing order.

Example:

```
[1, 3, 5, 12, 14]
```

Write a function that determines whether the user ever made more than 3 requests during any 10-second period. Return true if they exceeded the limit. Otherwise, return false.

Code Problem

Now the website has many users.

Each request contains:

```
user_id  
timestamp
```

For example:

```
{1, 1}  
{2, 2}  
{1, 4}  
{1, 6}  
{2, 7}  
{1, 8}
```

A user is allowed at most 3 requests during any 10-second period.

Process the requests one at a time. For every request, return true if the request should be accepted, or false if accepting it would cause that user to exceed the limit.

Each user's requests must be tracked independently.

Problem 3: Package Dependencies

Warm-up

You have software packages. Each package can depend on other packages.

For example:

```
A depends on B and C
B depends on D
C depends on nothing
D depends on nothing
```

Write a function that receives a package ID and returns the number of packages it directly depends on.

Example:

```
A -> 2
B -> 1
C -> 0
```

Code Problem

Now count all dependencies, including indirect dependencies. Using the same example:

```
A -> B
A -> C
B -> D
```

Package A depends on:

```
B
C
D
```

So the answer for A is:

```
3
```

A package should only be counted once, even if it can be reached through multiple paths.

For example:

```
A -> B
A -> C
B -> D
C -> D
```

D should still only be counted once.

Write a function that takes a package ID and returns the total number of unique direct and indirect dependencies.

Problem 4: Parking Garage

Warm-up

A parking garage records when each car enters and leaves.

Each visit is represented as:

```
[arrival_time, departure_time]
```

Example:

```
[1, 4]
[5, 8]
[3, 6]
```

Write a function that determines whether there was ever a time when two or more cars were inside the garage at the same time. Return true if this happened. Otherwise, return false.

Code Problem

Now determine the largest number of cars that were inside the garage at the same time.

Example:

```
visits = {
    {1, 6},
    {2, 5},
    {4, 8},
    {7, 10}
}
```

From time 4 to 5, the first three cars are all inside.

Therefore the answer is:

```
3
```

Return the maximum number of cars present at any single moment.

Problem 5: Customer Support Tickets

Warm-up

A customer support system receives tickets.

Every ticket has:

`customer_id`

Example:

4, 2, 4, 7, 2, 4

Write a function that returns how many unique customers submitted at least one ticket.

For the example above, the customers are:

2, 4, 7

So the answer is:

3

Code Problem

Now tickets are received one at a time.

Each ticket contains:

`customer_id`
`priority`

The priority is an integer where a smaller number means more urgent.

Example:

{101, 3}
{102, 1}
{103, 2}
{104, 1}

Your support system needs to process tickets in order of urgency.

Implement a system that supports these two operations:

`addTicket(customer_id, priority)`

Adds a new ticket.

`processNext()`

Removes and returns the customer ID of the most urgent ticket currently waiting.

If two tickets have the same priority, the ticket that was submitted earlier should be processed first.